

Classifieur hybride Qwen — récapitulatif des mises à jour

Projet Awalé Boissons · fichier `ai/classify_comments_hybrid.py` · 29 septembre 2026

1. En bref

Le checkup a montré que Qwen n'était pas le premier problème : ses erreurs venaient surtout de ce qu'on lui donnait en entrée (résultats de règles faux présentés comme définitifs, consignes contradictoires, prompt trop long pour un modèle de 0,5 milliard de paramètres) et d'une évaluation qui ne mesurait pas la bonne version. Après correction, l'accord exact sur les 5 dimensions du benchmark humain (50 commentaires) passe de **14 %** à **92 %**.

Dimension	Ancienne mesure (V2)	Après règles V3.1	Après V4 (actuel)
language	22 %	90 %	96 %
sentiment	80 %	46 %	96 %
theme	78 %	60 %	96 %
product	52 %	76 %	98 %
is_spam	88 %	98 %	100 %
Accord exact (5 dimensions)	14 %	26 %	92 %

Indicateur	Avant	Après
Commentaires envoyés à Qwen	2 831 / 2 831 (100 %)	377 / 2 831 (13 %), soit 197 textes uniques
Durée du benchmark (50 commentaires, CPU)	plus de 10 minutes	52 secondes
Tests automatisés du classifieur	cassés (signature V3)	21 tests, tous verts

Limite importante : sur ce benchmark, les règles tranchent désormais presque tout. Le score de 92 % mesure donc surtout les règles ; la qualité de Qwen sur les 13 % restants n'est pas encore mesurée par un benchmark humain (voir section 5).

2. Ce qui n'allait pas et la solution apportée

#	Problème constaté	Conséquence	Solution apportée
1	Le script appelait <code>test_v3()</code> au lieu de <code>main()</code> .	<code>run_pipeline.py</code> ne produisait plus aucune prédiction.	Le test devient une option : <code>--test N</code> (N commentaires annotés, détail règle / Qwen / humain) et <code>--bench</code> (tout le benchmark). Sans option : run complet.
2	Marqueurs « anglais » qui existent aussi en français : a, on, me, an, or, but, no, hey, can (CAN).	Faux « mixed » (« le prix a encore monté ») imposés à Qwen comme définitifs. Précision des règles de langue : 85 %.	Ces mots sont retirés des marqueurs anglais. Précision des règles de langue : 98 %.
3	Regex <code>ru[p pt]ure</code> (classe de caractères).	Le mot « rupture » n'était jamais détecté.	Corrigée en <code>\brupture\b</code> .

#	Problème constaté	Conséquence	Solution apportée
4	Consigne « produit non cité → none » contraire aux labels humains (42 % de « unknown »).	Qwen ne pouvait pas trouver le bon label ; il inventait aussi des produits (« bouye » 457 fois).	Le produit n'est plus demandé à Qwen : nommé → règle ; sinon « unknown » si le thème porte sur la boisson (goût, prix, stock, emballage, santé, question produit), « none » sinon.
5	L'évaluation lisait un fichier de prédictions du 17/09.	Les scores affichés ne concernaient pas la version en cours.	--bench régénère model_predictions_hybrid.csv avec la version courante avant chaque évaluation.
6	Le fichier de sortie contenait déjà les 2 831 prédictions V2 et le script saute les commentaires déjà classés.	Un run complet n'aurait rien reclassé.	Colonne classifier_version (règles + prompt + modèle) : les lignes d'une autre version sont sauvegardées en .backup_<date>.csv puis reclassées.
7	Prompt V2 long, avec un JSON d'exemple rempli (fr / neutral / other / none) et des consignes répétées dans le message utilisateur.	Qwen recopiait l'exemple ou répondait presque toujours « negative » (sentiment 46 %).	Prompt V4 (nouveau fichier) : court, placeholders, 12 exemples few-shot en tours de conversation, aucun tiré du benchmark (vérifié par un test). Qwen reçoit le commentaire seul.
8	Règles trop pauvres : sentiment décidé pour 9 commentaires sur 50, thème pour 19.	Qwen faisait l'essentiel du travail, sur les cas où il est le plus faible.	Règles enrichies (voir 3.1). Le modèle n'est plus appelé que pour 13 % du corpus.
9	device_map="auto" sur CPU ; accelerate absent de l'environnement.	Plantage au chargement, puis poids partiellement déchargés sur disque (très lent).	Chargement direct sur CPU (float32) ou GPU (bfloat16). accelerate installé à la version des requirements.
10	45 % de doublons exacts dans le corpus (1 551 textes uniques / 2 831).	Calcul inutile, et deux textes identiques pouvaient recevoir des labels différents.	Cache par texte normalisé : chaque texte n'est envoyé qu'une fois à Qwen.
11	Signature du modèle modifiée en V3 (rules_batch) ; THEME_RULES et model_classification jamais utilisés.	Tests existants cassés ; code mort divergent.	Signature (texts, tokenizer, model) rétablie, code mort supprimé, logique de fusion centralisée dans merge_prediction.

3. Détail des améliorations

3.1 Règles déterministes (RULES_VERSION = v4)

- **Sentiment** : adorent, doux, enjaillant, (y a) pas drap, ne regrette pas, nickel, joli, meilleur, « c'est propre / frais », bien fait / placé / organisé... ; négatifs : introuvable, rupture, stock fini, plus rien, annulée, lente, ne répond pas, se décolle, baisser / diminuez le prix, encore monté...
- **Question sans marqueur d'avis** → **neutral** (« vous livrez à Bingerville ? »).
- **Thèmes** : vocabulaire enrichi (33cl / 1L / packs → packaging, livrez / livreur → delivery, supermarché / introuvable → availability, payer / wave → service...) et priorités quand deux thèmes se chevauchent (product_question > health > taste ; promotion, price > taste ; delivery > availability).
- **Spam** : vente (« vends »), bit.ly, « cliquez ici », trading, perte de poids, numéros de téléphone, follow-for-follow. Un spam reçoit sentiment neutral, thème other, produit none.
- **Nouchi** : tchè, cé, gbê / gbé, kpakpato, drap, môgô en plus des termes existants.
- **Cas sans modèle** : commentaires en emojis seuls, formules de conversation (« bonjour », « +1 », « je confirme », « exact »), « bon / mais / eh » en début de phrase ignorés.

3.2 Prompt V4 — ai/prompts/comment_classifier_v4_hybrid.txt

- Consignes courtes, définitions de chaque label, JSON attendu avec des placeholders.
- 12 exemples après la ligne « `=== EXEMPLES ===` », envoyés comme des échanges utilisateur / assistant.
- Qwen ne renvoie que 4 champs (language, sentiment, theme, is_spam) ; les règles priment à la fusion.

3.3 Tests ajoutés (tests/test_classify_checkpoint.py)

- Mots français non pris pour de l'anglais ; « rupture » détecté ; produit déduit du thème et jamais pris au modèle ; emojis seuls sans modèle ; spam neutre ; doublons envoyés une seule fois au modèle ; exemples few-shot valides et absents du benchmark.

4. Vérifier la précision et l'accuracy sur les 5 dimensions

Toutes les commandes se lancent depuis la racine du projet (awale_boissons/).

Étape 0 — Environnement

```
pip install -r requirements.txt      # torch, transformers, accelerate
pip install -r requirements-dev.txt  # pytest
```

Étape 1 — Tests de la logique (sans modèle, quelques secondes)

```
python -m pytest -q tests/test_classify_checkpoint.py
```

Attendu : 21 tests réussis.

Étape 2 — Contrôle rapide sur quelques commentaires annotés

```
python ai/classify_comments_hybrid.py --test 10
```

Affiche, pour chacun des 10 premiers commentaires du benchmark, la valeur de la règle, la réponse de Qwen, la valeur finale et le label humain (OK / XX), puis l'accuracy par dimension. N'écrit aucun fichier.

Étape 3 — Benchmark complet (50 commentaires annotés, ~1 min sur CPU)

```
python ai/classify_comments_hybrid.py --bench
```

Écrit ai/evaluation/model_predictions_hybrid.csv avec la version courante et affiche l'accuracy par dimension.

Étape 4 — Précision, rappel, F1 et accuracy sur les 5 dimensions

```
python ai/evaluation/evaluate_classifier.py --model hybrid
```

Affiche pour language, sentiment, theme, product et is_spam : l'accuracy et le rapport scikit-learn par classe (precision, recall, f1-score, support), puis l'accord exact sur les 5 dimensions. Les commentaires avec au moins une erreur sont écrits dans ai/evaluation/classification_errors_hybrid.csv.

Seuil de référence (version actuelle)	Valeur mesurée le 29/09/2026
Accuracy par dimension	96 à 100 %
Accord exact sur les 5 dimensions	92 % (4 commentaires en erreur sur 50)
Commentaires en erreur	2 dont le label humain est discutable (voir 5.1)

Étape 5 — Couverture des règles sur le corpus complet (sans modèle)

```
python -c "  
import sys; sys.path.insert(0, 'ai')  
import pandas as pd  
import classify_comments_hybrid as h  
d = pd.read_csv(h.INPUT_FILE).comment_text.astype(str)  
m = ~d.map(h.deterministic_classification).map(h.is_complete)  
print('Envoyés à Qwen :', m.sum(), '/', len(d))  
print('Textes uniques :', d[m].map(h.normalize_text).nunique())  
"
```

Attendu : 377 / 2 831 commentaires, 197 textes uniques.

5. Ce qu'il reste à faire avant le run sur les 2 831 commentaires

5.1 Obligatoire

#	Action	Pourquoi
1	Trancher deux labels du benchmark : « j'ai fait 3 boutiques, rien qui est comme moi ? » (humain : positive / other, règles : negative / availability) et « stock fini... dèh » (humain : fr / neutral, alors que la même phrase sans « dèh » est negative).	Un benchmark incohérent fausse la mesure et pousse à « corriger » des règles justes.
2	Agrandir le benchmark avec 50 à 100 commentaires que les règles ne tranchent pas (« wê on veut le gingembre », « j'ai vu chez @... », « le 33cl à 500 c'est correct mais... »), annotés à la main, puis relancer les étapes 3 et 4.	Aujourd'hui, rien ne mesure la qualité réelle de Qwen sur les 13 % qui lui reviennent. Constat qualitatif : réponses instables selon un emoji ajouté.
3	Vérifier l'environnement (étape 0) et lancer les tests (étape 1).	Le run échoue sans accelerate ; les tests valident reprise, sauvegarde et garde-fous.
4	Commiter le code, le prompt V4 et les tests avant le run.	La colonne classifier_version renvoie à une version précise : elle doit exister dans git.
5	Décider du sort de comment_classifier_v2_hybrid.txt (modifications non committées, fichier plus utilisé) et des scripts ai/test*_v3.py non suivis.	Éviter qu'un ancien prompt modifié soit pris pour la référence.

5.2 Recommandé

- Réduire les exemples few-shot de 12 à 6–8 : Qwen prend environ 11 s par texte sur CPU (197 textes ≈ 36 min). Relancer le benchmark après toute modification du prompt (nouveau fichier v5).
- Mettre à jour docs/ai_documentation.ipynb et le Runbook mensuel, qui décrivent encore le prompt V2.
- Plusieurs exemples du prompt ressemblent à des textes que Qwen doit classer (« vu votre pub », « gbé ») : garder cela en tête en lisant ses résultats.

5.3 Lancer le run complet puis vérifier

```
python ai/classify_comments_hybrid.py
```

- Durée estimée : environ 40 minutes sur CPU (197 textes uniques pour Qwen, le reste par les règles). Une interruption ne fait rien perdre : relancer la même commande reprend là où le run s'est arrêté.
- Les 2 831 anciennes prédictions V2 sont sauvegardées automatiquement dans social_comments_predictions_v2_full.backup_<date>.csv puis reclassées.
- En fin de run : aucune ligne **[ERREUR]** (sinon relancer) ; surveiller **[ATTENTION]** (réponses hors vocabulaire remplacées).

Contrôle du fichier produit :

```
python -c "
import pandas as pd
p = pd.read_csv('ai/evaluation/social_comments_predictions_v2_full.csv')
print(len(p), 'lignes | ids uniques :', p.comment_id.is_unique)
print('versions :', p.classifier_version.unique())
for c in ['language', 'sentiment', 'theme', 'product', 'is_spam']:
    print(c, p[c + '_model'].value_counts().to_dict())
"
```

Attendu : 2 831 lignes, identifiants uniques, une seule version. Comparer les distributions avec l'ancien fichier (backup) : l'ancienne version classait 2 509 commentaires en « en » et 457 en « bouye », ce qui ne doit plus apparaître.

Puis chargement et transformation :

```
python ingestion/load_predictions.py
cd dbt
dbt run
dbt test
cd ..
```

Ou directement l'ensemble du cycle : `python run_pipeline.py`.